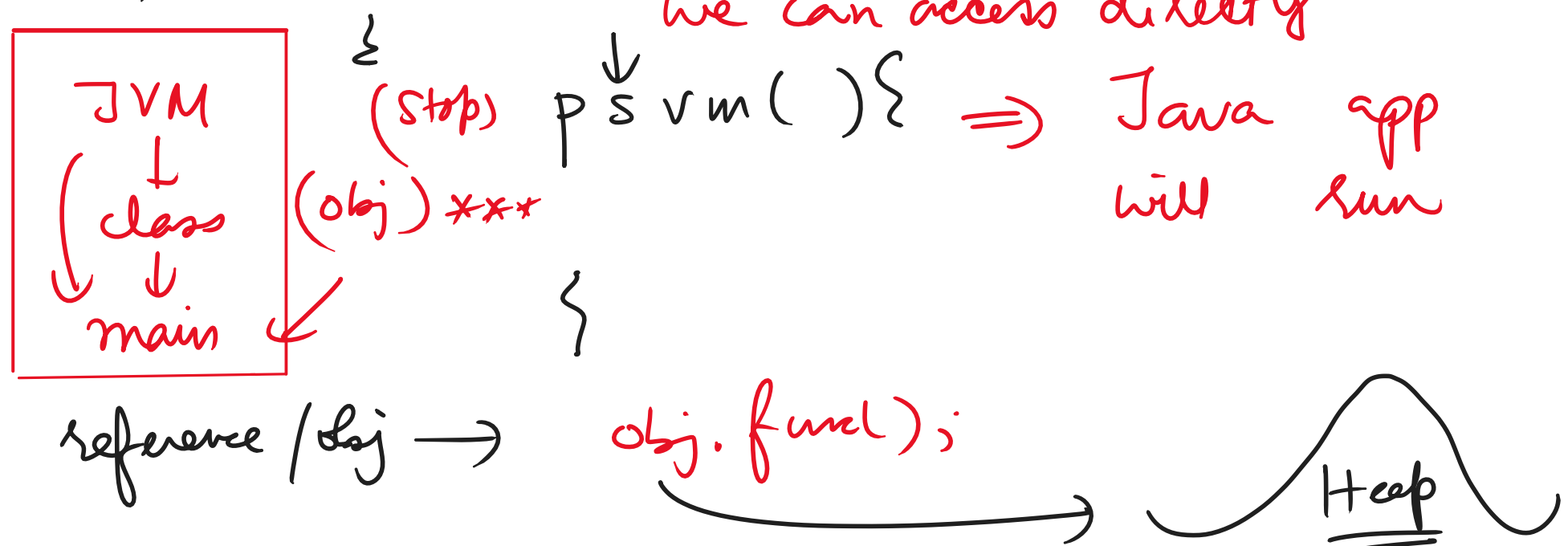


- * Static variables are shared by all objects
- * Instance variables are object-specific.
- * Static variables are "Memory Efficient" for common data for the whole class.

→ v fune() {



* Can we start Java App without main()

⇒ Yes using static block but till J 1.6

But from 1.7 main is mandatory

Static → class

1. Static method works on static data
2. No object is needed to call static method
3. Tracks class-level Information.

4 Pillars of OOPS: →

* Encapsulation: → Binding / Wrapping the data members & the methods inside a block of code { class }, so that they are not accidentally modified, is called data encapsulation. { Safety }

* It is achieved by using the "private" access modifier.

* To access outside the class we use special public methods →

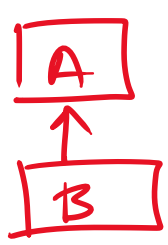
Setters → assign attributes
↳ void

Getters → fetch attributes
↳ return type

Inheritance: → The property by virtue of which a child class can fetch data from parent classes, is called inheritance.

Benefits: ① Code Reusability
② Memory Efficiency Increases

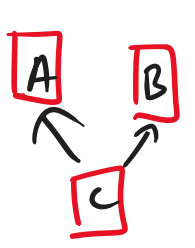
① Single Level



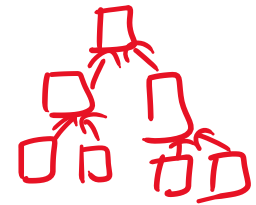
② Multi level



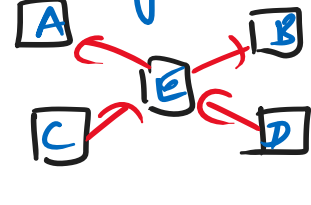
③ Multiple



④ Hierarchical



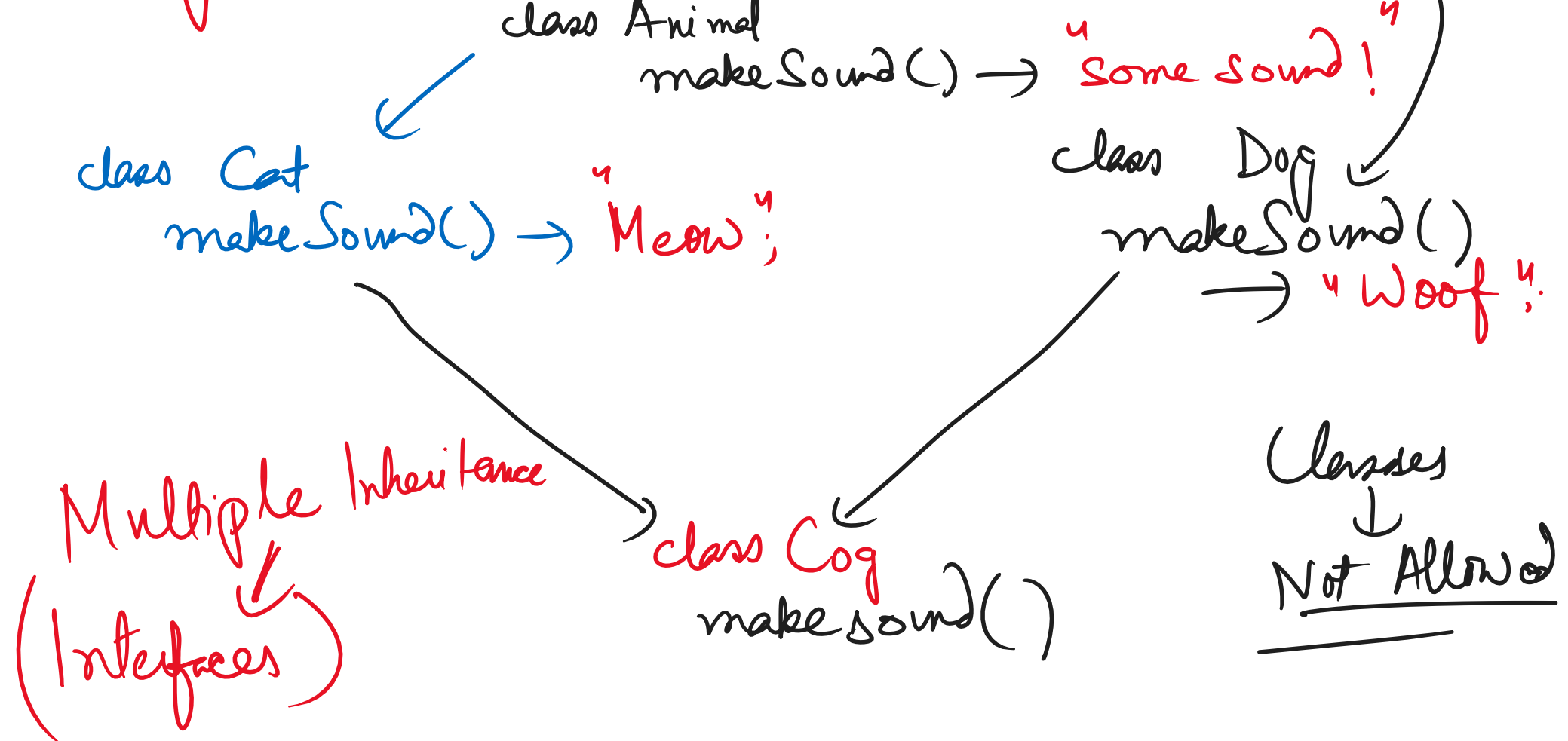
⑤ Hybrid



* Can we extend multiple classes in Java?

⇒ No

* Why: → Diamond Problem:



Hierarchical Inheritance:

Abstraction: →

Hiding the implementation details & only showing the functionality to the

"end user" is called Data Abstraction.

It is done for better (UX) ⇒ user experience.
Or, else the customer will get confused & leave the application.

Two ways:

- (i) Abstract classes & methods (0 - 100%)
- (ii) Interfaces (100%)

(implementation)
Hiding → How it is happening
Showing → What is happening (functionality)